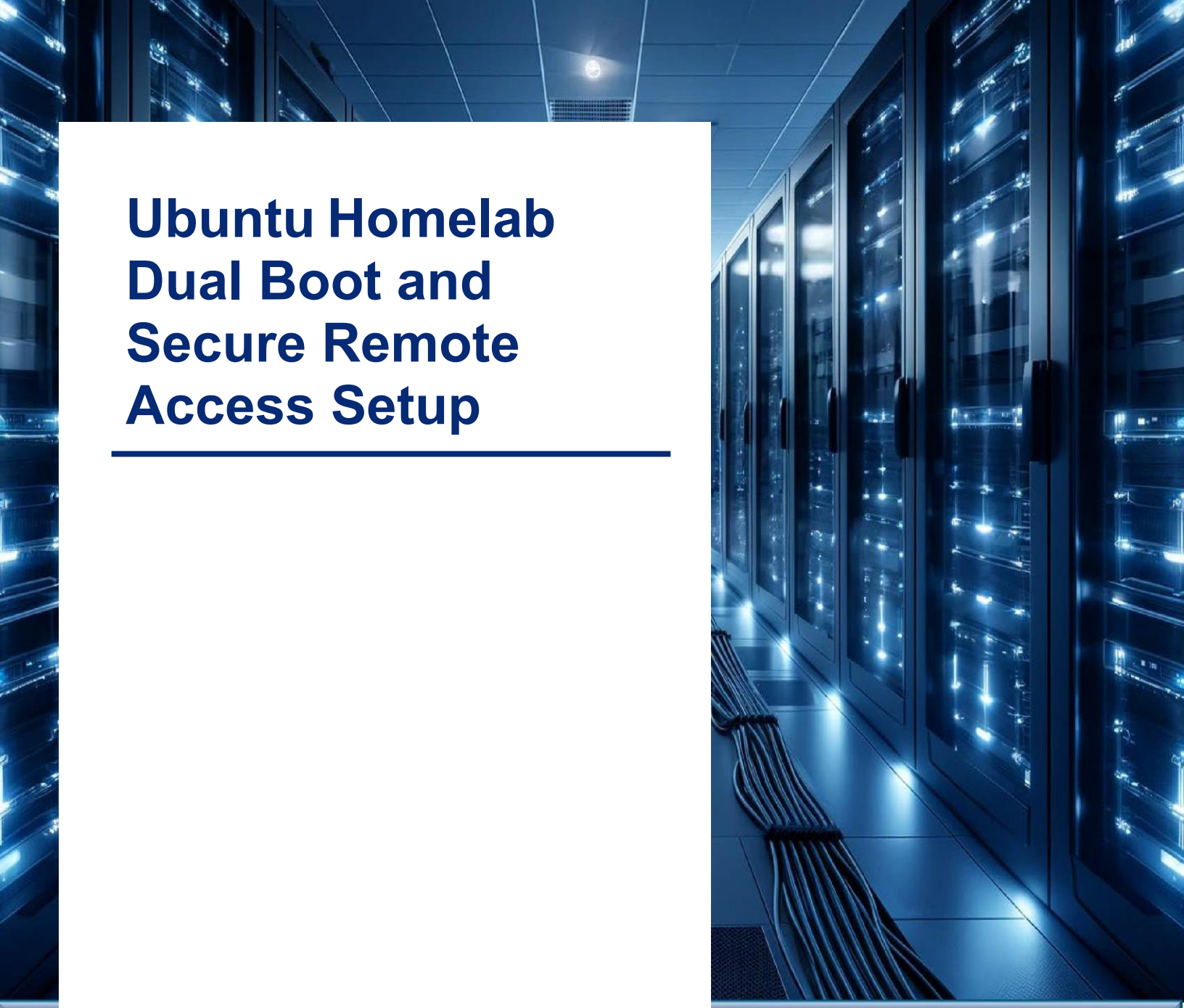

Ubuntu Homelab Dual Boot and Secure Remote Access Setup

June 2025

V1.0

Authored by: Niv Reddy

Creating a Dual Boot Setup (Ubuntu + Windows)

1.1 Preparing for Installation

- Backed up any important data
- Downloaded Ubuntu ISO from the official Ubuntu website
  Ubuntu 24.0.4 LTS
- Used Rufus to create a bootable USB

1.2 Installing Ubuntu alongside Windows

- Plugged in the bootable USB drive
- Accessed BIOS/UEFI
- Selected the USB drive to boot from
- Ubuntu Installer:
  - “Install Ubuntu alongside Windows Boot Manager”
  - Allocated appropriate disk space for Ubuntu
  - "Minimal installation”
  - Enabled third-party drivers and media support

1.3 Post-Installation Setup

- Rebooted and Confirmed GRUB menu allows booting both Ubuntu and Windows

SSH Setup for Remote Access

2.1 SSH Key Generation

To enable secure and passwordless remote access from the Mac to the Ubuntu homelab, we used SSH key-based authentication instead of traditional password login. This improves both security and convenience.

Why Key-Based SSH?

- More secure than passwords: SSH keys use strong cryptographic algorithms that are extremely difficult to brute-force, unlike typical login passwords.
- No need to type password every time: Once set up, SSH key authentication allows instant access without having to input the Ubuntu user's password.
- Better for automation and scripting: Many homelab tasks (like remote backups or scripts) benefit from seamless SSH access.

- Generated a new SSH key pair on the Mac:

```
ssh-keygen -t ed25519 -C "remote access to Ubuntu homelab
```

`-t ed25519`: Specifies the modern and secure Ed25519 algorithm for key generation.
`-C`: Adds a comment to label the key (useful when managing multiple keys).

Copied Public Key to the Ubuntu Machine:

- This is what enables the Mac to authenticate with Ubuntu.
- The public key was added to Ubuntu's ~/.ssh/authorized_keys file for the nreddy user.
- Once copied, the Mac's private key is used silently during SSH to prove identity.

2.2 SSH Server Setup on Ubuntu

- Installed and enabled OpenSSH server:

  ```
  sudo apt update && sudo apt install openssh-server
  sudo systemctl enable ssh
  sudo systemctl start ssh
  ```

- Changed SSH port in /etc/ssh/sshd_config:

  ```
  Port <custom-port>
  ```

  I changed the default SSH port from 22 to a non-standard port to reduce exposure to automated attacks and bot scans. Port 22 is commonly targeted on the internet, so switching to a custom port adds a basic layer of security through obscurity.

- Reloaded daemon and restarted SSH:

  ```
  sudo systemctl daemon-reexec
  sudo systemctl restart ssh
  ```

2.3 Port Forwarding via Eero

- Logged into Eero portal.
- Set up port forwarding:
  - Forwarded external port to internal port on the Ubuntu machine's local IP.
  - Used my-eero-domain.eero.onlinedomain provided by Eero for remote access.

Secure SSH Configuration

To harden the SSH setup and reduce the attack surface, the following configurations were made in the /etc/ssh/sshd_config file:

```
Port <custom-port>                  # Changed default port to avoid automated bot scans
PermitRootLogin no                  # Disabled root login for safety
PubkeyAuthentication yes            # Ensures only key-based authentication is allowed
PasswordAuthentication no           # Disable password login to prevent brute-force attacks
KbdInteractiveAuthentication no     # Disables interactive keyboard auth
UsePAM yes                          # Keeps PAM enabled for session control and logging
X11Forwarding yes                   # Allows GUI apps forwarding if needed
PrintMotd no                        # Suppresses Message of the Day on login
AllowUsers <username>               # Restricts SSH access to a specific user
```